Linux demo commands

OSNT Project 16 | Run one command at a time in Bash. Open a terminal in the updated OSNT_Project folder first.

Page 1: prepare before class. Use your normal Linux account; enter your password only when sudo requests it. Keep the same terminal open.

01 Confirm the project folder

```
pwd
```

Show the current folder. **Expect:** A path ending in OSNT_Project; README.md, topology/ and results/ must be here.

02 Refresh package information

```
sudo apt update
```

Refresh Ubuntu package indexes. **Expect:** Package lists finish successfully. Internet access is needed.

03 Install the C build tools

```
sudo apt install -y build-essential cmake ninja-build git libssl-dev
```

Install the compiler, build system, Git and OpenSSL headers. **Expect:** Packages installed, or already at the newest version.

04 Install Python and network tools

```
sudo apt install -y python3-venv iproute2 iputils-ping xdg-utils
```

Install virtual environments, Linux namespace tools, ping and desktop file opening. **Expect:** Packages installed, or already present.

05 Create the Python environment

```
python3 -m venv .venv
```

Create a project-local Python environment. **Expect:** Usually no terminal output.
Saved: .venv/ under the project root.

06 Install the plotting dependency

```
.venv/bin/python -m pip install -r experiments/requirements.txt
```

Install matplotlib in that environment. **Expect:** Installation succeeds, or Requirement already satisfied.
Saved: .venv/; no saved experiment is changed.

07 Configure picoquic

```
cmake -S . -B build -G Ninja -DPICOQUIC_FETCH_PTLS=Y -DWITH_OPENSSL=ON -DCMAKE_BUILD_TYPE=RelWithDebInfo
```

Generate the Linux build and fetch Picotls if needed. **Expect:** Configuring done, Generating done, and build files written to build/.
Saved: build/; the first configuration needs Internet access.

08 Build the executables

```
cmake --build build --parallel 4
```

Compile picoquic and its tests. **Expect:** Successful build, or ninja: no work to do.
Saved: build/picoquicdemo and build/picoquic_ct.

09 Run the registered regression suites

```
sudo ctest --test-dir build --output-on-failure
```

Check both registered test runners. **Expect:** 100% tests passed, 0 tests failed out of 2. Runtime varies.
Saved: build/Testing/Temporary/LastTest.log. Stop and investigate any failure.

Before class, confirm that results/run_single_comparison.sh exists. These local updates must be copied or pushed before using a different Linux checkout.

Live demonstration

Page 2 | Start here after setup. The two fixed-rate runs take about 20 seconds each. Wait for each command to finish.

This is an RLI-off/on demonstration using the same current binary. The report's historical baseline used a separately built pre-RLI revision.

10 Show the focused RLI test

```
./build/picoquic_ct cubic_rli
```

Exercise the RLI implementation checks. **Expect:** Starting test ... cubic_rli, followed by Success.

11 Create the test network

```
sudo bash topology/setup.sh
```

Create the four isolated client/router/server namespaces. **Expect:** Dumbbell topology created; 20 Mbit/s and about 50 ms base RTT.

12 Verify the network

```
sudo bash topology/check.sh
```

Ping across the topology and display the bottleneck settings. **Expect:** 0% packet loss; netem delay 25ms rate 20Mbit in both directions.

13 Run the baseline with RLI disabled

```
sudo env PYTHON_BIN="$PWD/.venv/bin/python" bash results/run_single_comparison.sh baseline rate-limited
```

Send 400 frames over about 20 seconds with RLI off. **Expect:** A plot path and Experiment completed. Typical cwnd is about 15 KiB.
Saved: experiments/results/baseline/rate-limited/<UTC-run-id>/

14 Run the same workload with RLI enabled

```
sudo env PYTHON_BIN="$PWD/.venv/bin/python" bash results/run_single_comparison.sh rli rate-limited
```

Repeat with RLI on. The terminal may stay quiet during the transfer. **Expect:** cwnd reaches about 30 KiB; active cap samples, PASS, and Experiment completed.
Saved: experiments/results/rli/rate-limited/<UTC-run-id>/

15 Remember the latest baseline folder

```
BASE=$(ls -dt experiments/results/baseline/rate-limited/*/ | head -n 1)
```

Store the latest baseline run path in a Bash variable. **Expect:** No output; keep this terminal open.

16 Remember the latest RLI folder

```
RLI=$(ls -dt experiments/results/rli/rate-limited/*/ | head -n 1)
```

Store the latest RLI run path in a Bash variable. **Expect:** No output; the following commands use this variable.

17 Display the baseline graph

```
xdg-open "$BASE/cubic-baseline-rate-limited.png"
```

Open the RLI-off congestion-window plot. **Expect:** An image viewer shows the baseline cwnd.
Saved: Graph is already in the folder stored in $BASE.

18 Display the RLI graph

```
xdg-open "$RLI/cubic-rli-rate-limited.png"
```

Open the RLI-on plot for comparison. **Expect:** cwnd grows toward 30 KiB and is capped; maxFS and the limit are plotted.
Saved: Graph is already in the folder stored in $RLI.

19 Show the run validation result

```
cat "$RLI/validation.txt"
```

Print the checks performed by the bundled campaign validator. **Expect:** PASS and sample/growth/cap counters. This is not every newer validation check.
Saved: validation.txt inside $RLI.

Results and optional bulk test

20 Show stream completion

```
grep "received 400/" "$RLI/client.stdout.log"
```

Show the QUICPERF frame completion summary. **Expect:** received 400/ 400 frames, followed by delay measurements.
Saved: client.stdout.log inside $RLI.

21 Open the complete RLI run folder

```
xdg-open "$RLI"
```

Open the fresh run in the Linux file manager. **Expect:** The graph, performance CSVs, qlogs, stdout and metadata are visible.
Saved: The folder path is the value of $RLI.

What to show in that folder

File	What it contains
cubic-rli-rate-limited.png	Congestion-window, FlightSize, maxFS and cap plot.
server-performance.csv	Sender bytes, rate, RTT and retransmission statistics.
client.stdout.log / server.stdout.log	Connection progress and normal process exit messages.
*.server.qlog / *.client.qlog	Fresh transport traces for this connection.
validation.txt / metadata.txt	Observed checks, source identity and executable hash.

Optional: demonstrate a 20 MiB file transfer

Keep the topology running for these commands. Each bulk transfer takes roughly 10-15 seconds on this configured path; exact results vary.

22 Run the bulk baseline

```
sudo env PYTHON_BIN="$PWD/.venv/bin/python" bash results/run_single_comparison.sh baseline bulk
```

Download the 20 MiB payload with RLI off. **Expect:** A plot path and Experiment completed.
Saved: experiments/results/baseline/bulk/<UTC-run-id>/; file_20M.bin is generated at the project root if absent.

23 Run the bulk RLI case

```
sudo env PYTHON_BIN="$PWD/.venv/bin/python" bash results/run_single_comparison.sh rli bulk
```

Repeat the bulk workload with RLI on. **Expect:** A plot, a validation result and Experiment completed.
Saved: experiments/results/rli/bulk/<UTC-run-id>/

24 Open the latest bulk RLI output

```
xdg-open "$(ls -dt experiments/results/rli/bulk/*/ | head -n 1)"
```

Open the bulk run folder to inspect its graph and performance CSV. **Expect:** The latest bulk RLI result folder opens.

25 Always clean up after the demo

```
sudo bash topology/cleanup.sh
```

Remove the project network namespaces, including any supplementary pair. **Expect:** Deleted rls-client, rls-r1, rls-r2 and rls-server for an active main topology.
Saved: Saved result files remain on disk.

Presentation point: RLI allows bounded growth from about 15 to 30 KiB in the limited-sender case. The fixed application supply keeps throughput near 0.19 Mbit/s. A larger cwnd alone is not a speedup.

If a command fails, stop rather than assuming success. Run command 25 to clean the network. Linux dependencies and administrative privileges are required; archived experiment logs are not runtime inputs.

Quick rerun after preparation

Keep the same Bash terminal open in the OSNT_Project root. The build/ executables and .venv/ must already exist. No installation, rebuild, or full regression rerun is needed for this demonstration.

Copy commands 26-32 in order. The two traffic commands start picoquic server/client processes, send fresh traffic, check the run, and generate the graphs automatically. Allow about one minute overall.

26 Run the picoquic RLI test

```
./build/picoquic_ct cubic_rli
```

Show the focused RLI implementation test. **Expect:** Starting test ... cubic_rli, then Success.

27 Recreate the network for this run

```
sudo bash topology/setup.sh
```

Create a clean four-namespace test path, even after an earlier cleanup. **Expect:** Dumbbell topology created; 20 Mbit/s and about 50 ms base RTT.

28 Generate baseline traffic and its graph

```
sudo env PYTHON_BIN="$PWD/.venv/bin/python" bash results/run_single_comparison.sh baseline rate-limited
```

Run picoquic with RLI off; send 400 frames over about 20 seconds. **Expect:** Saved plot and Experiment completed. Typical cwnd stays near 15 KiB.

Saved: experiments/results/baseline/rate-limited/<new-UTC-run-id>/

29 Generate RLI traffic and its graph

```
sudo env PYTHON_BIN="$PWD/.venv/bin/python" bash results/run_single_comparison.sh rli rate-limited
```

Repeat with RLI on. Wait for the full command to finish. **Expect:** cwnd rises toward 30 KiB; active cap samples, PASS, then Experiment completed.

Saved: experiments/results/rli/rate-limited/<new-UTC-run-id>/

30 Open the newest baseline graph

```
xdg-open "$(ls -t experiments/results/baseline/rate-limited/*/*.png | head -n 1)"
```

Display the graph generated by the latest baseline run. **Expect:** The baseline congestion-window plot opens in an image viewer.

Saved: PNG in the newest baseline run folder above.

31 Open the newest RLI graph

```
xdg-open "$(ls -t experiments/results/rli/rate-limited/*/*.png | head -n 1)"
```

Display the latest RLI graph without reusing an old folder variable. **Expect:** Compare the bounded window growth with the baseline graph.

Saved: PNG in the newest RLI run folder above.

32 Clean up when finished

```
sudo bash topology/cleanup.sh
```

Remove the temporary project network namespaces. **Expect:** Deleted namespace messages. All graphs and run results remain saved.

What to explain: "We first check the RLI implementation. Then we run the same fixed-rate workload with RLI off and on. The RLI window grows within its cap; the application supply keeps throughput near 0.19 Mbit/s."

Repeat again: start at command 26. Every traffic run creates a new timestamped folder. The graph commands select the newest PNG, so the earlier BASE and RLI shell variables are not required.

The live baseline is RLI off in the same current binary. The report's historical baseline was a separately built pre-RLI revision. If any command fails, stop and use command 32 for cleanup.